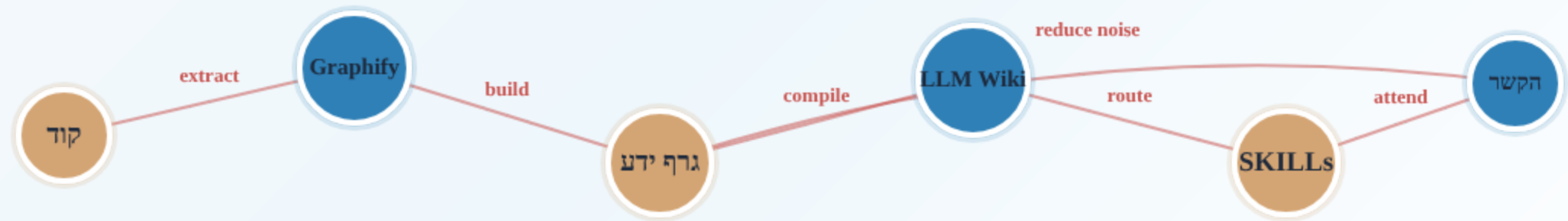


ארכיטקטורות ידע מבוססות גרפים וניהול הקשר במודלי שפה גדולים

מ־Graphify ו־LLM Wiki ועד Claude SKILLs ו־Lost in the Middle



למה המשך המצגת מתחיל בהקשר?

הכשל המרכזי אינו רק "חוסר ידע"; הוא ניהול לא יעיל של מה נכנס לקשב של המודל.

גרף ידע מצמצם קוד ומסמכים למבנה שניתן לנווט בו.

LLM Wiki מחזיק זיכרון מעובד במקום מסמכים גולמיים.

SKILLS מגדירים לסוכן פרוטוקולים קצרים להפעלת כלים.



הגדרות: **Context Window** = חלון ההקשר שהמודל רואה; **Attention** = קשב
חישובי בין טוקנים; **Signal-to-Noise** = יחס אות-לרעש.

Graphify הופך קוד למפת ידע

במקום לקרוא תיקיות קוד כטקסט ארוך, Graphify מייצר מפה מבנית שניתנת לשאילה ולניווט.

פונקציות, מחלקות ומודולים הופכים לצמתים.

קריאות, תלות והסברים הופכים לקשתות בעלות משמעות.

הפלט כולל `graph.json`, `graph.html` ודוח ניתוח.

דיווחי שימוש מציינים עד 71.5× פחות טוקנים בשאילות קוד.

קוד גולמי → גרף ידע



המודל שואל את המבנה במקום לסרוק את כל הקבצים

הגדרות: Node = צומת, יחידת ידע; Edge = קשת, קשר בין יחידות; Graph = רשת צמתים וקשתות.

RAG נאיבי מכניס רעש לחלון

ב־RAG נאיבי, המודל מקבל הרבה טקסט “כמעט רלוונטי” – ולא בהכרח את הידע הנכון.

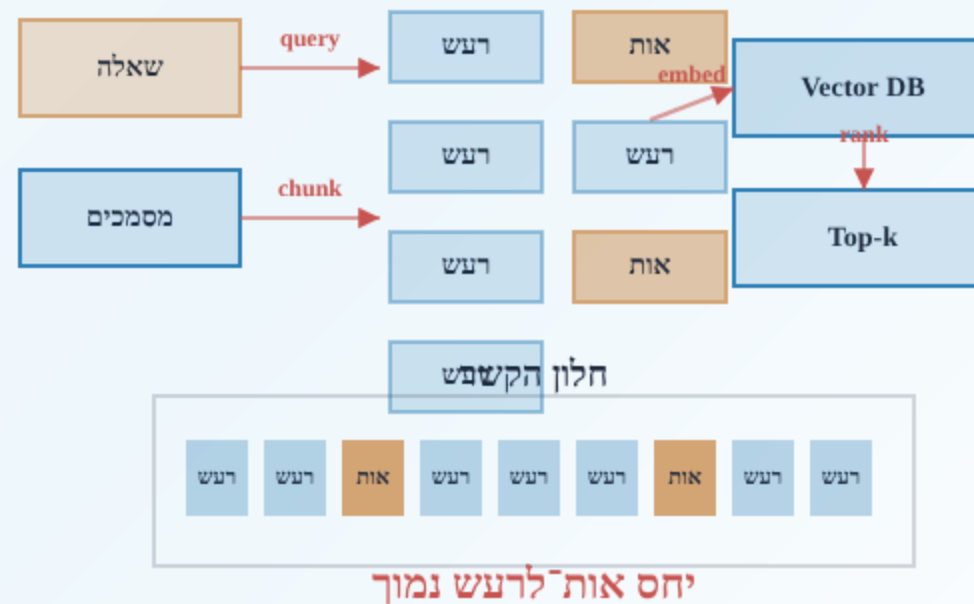
Chunking מפרק מסמך למקטעים ומוחק חלק מההיררכיה המקורית.

Vector Retrieval מחזיר קטעים דומים, אך דמיון אינו זהה לרלוונטיות.

חלון ההקשר מתמלא במסיחים, ולכן משאב ה־**Attention** מתפזר.

הבעיה אינה רק כמה טוקנים נכנסים – אלא כמה מהם הם אות ולא רעש.

שליפה נאיבית: הרבה מקטעים, מעט אות



מילון קצר: RAG = שליפה מוגברת יצירה; **Chunking** = פיצול מסמך למקטעים; **Vector DB** = מסד וקטורי למציאת דמיון סמנטי; **Noise** = מידע מסיח שאינו תורם לתשובה.

הרעיון של קרפתי: לא לשלוף הכול מחדש – אלא לקמפל ידע גולמי לזיכרון מקושר ומתוחזק.

raw docs הם מסמכי מקור גולמיים: ארוכים, כפולים, ולעיתים רועשים.

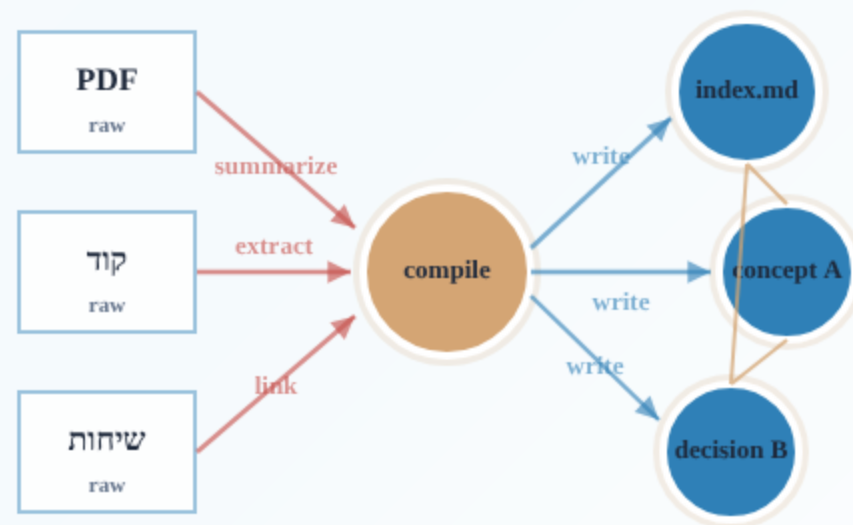
wiki pages הם דפי ידע קצרים: מסוכמים, מקושרים, וקריאים גם לאדם.

index.md משמש כ־**Hub** ניווט: קוראים אותו קודם, ואז בוחרים רק את הדפים הנחוצים.

כך ההקשר מתמלא בידע רלוונטי, לא בערימת טקסט שנשלפה במקרה.

מונח: Compilation בהקשר זה פירושו עיבוד, סיכום וקישור של מידע גולמי לצורת ידע יציבה.

ממסמכים גולמיים לזיכרון מקושר



קוראים אינדקס קטן, לא ארכיון שלם

הגדרה: LLM Wiki = ויקי קבצי **Markdown** שהמודל יוצר, מעדכן ומקשר, כדי לשמור ידע מעובד במקום לשלוף שוב מסמכים גולמיים.

מבנה מינימלי של ארבעה רכיבים הופך מסמכים גולמיים לזיכרון שניתן לקרוא, לעדכן ולנווט.

raw/ — מאגר קלט גולמי: PDF, הערות, קוד, תמלילים ומסמכי מקור שלא עברו עיבוד.

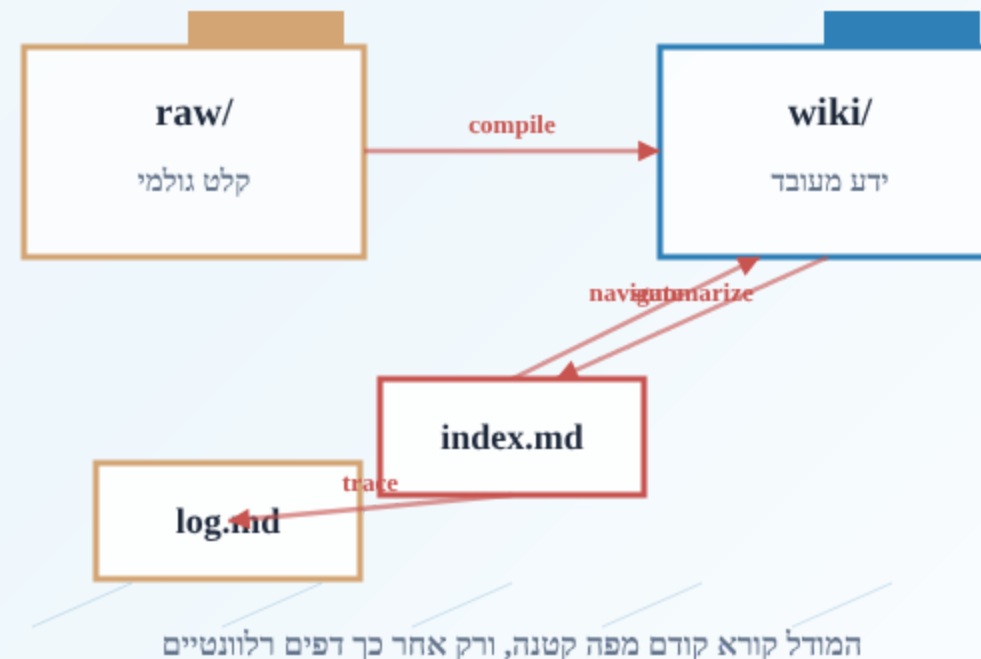
wiki/ — שכבת ידע מעובדת: דפי Markdown קצרים, מקושרים, ומנוסחים עבור שליפה חוזרת.

index.md — **צומת ניווט**: מפה קומפקטית שמכוונת את המודל לדפי הידע הרלוונטיים.

log.md — **עקיבות**: יומן שמסביר מה נקלט, מתי, ומה השתנה בזיכרון.

Markdown = פורמט טקסט פשוט עם כותרות וקישורים; **Hub** = צומת מרכזי שממנו יוצאים לקשרים רבים.

מערכת קבצים שהיא גם גרף ידע



העיקרון: הידע אינו נשמר כערימת מסמכים, אלא כמערכת קבצים קטנה שבה לכל שכבה יש תפקיד ניווטי ברור.

Guided Retrieval מצמצם את ההקשר

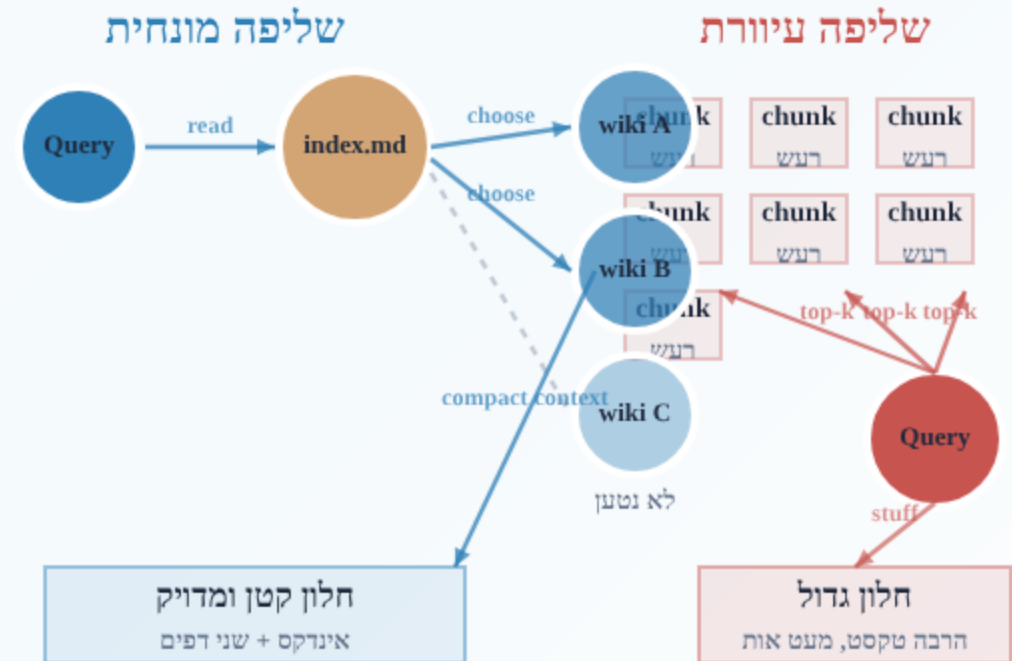
במקום "לזרוק מסמכים להקשר", המודל מנווט דרך אינדקס.

RAG נאיבי מחזיר הרבה קטעים דומים, אך לא תמיד חשובים.

`index.md` פועל כמו צומת מרכזי: הוא מכוון את המודל לדפים הנכונים.

התוצאה היא חלון הקשר קטן יותר, נקי יותר, ועם יחס אות-לרעש גבוה יותר.

כלל עבודה: שאלה → אינדקס → 2-3 דפי `wiki` → תשובה. לא כל המאגר נכנס להקשר; רק המסלול הרלוונטי.



הגדרה: Guided Retrieval = שליפה מונחית: קודם קוראים אינדקס קטן, ורק אחר כך טוענים מספר מצומצם של דפי ידע רלוונטיים.

Skeleton + Wiki יוצרים זיכרון דו־שכבתי

Graphify שכבת שלד קוד

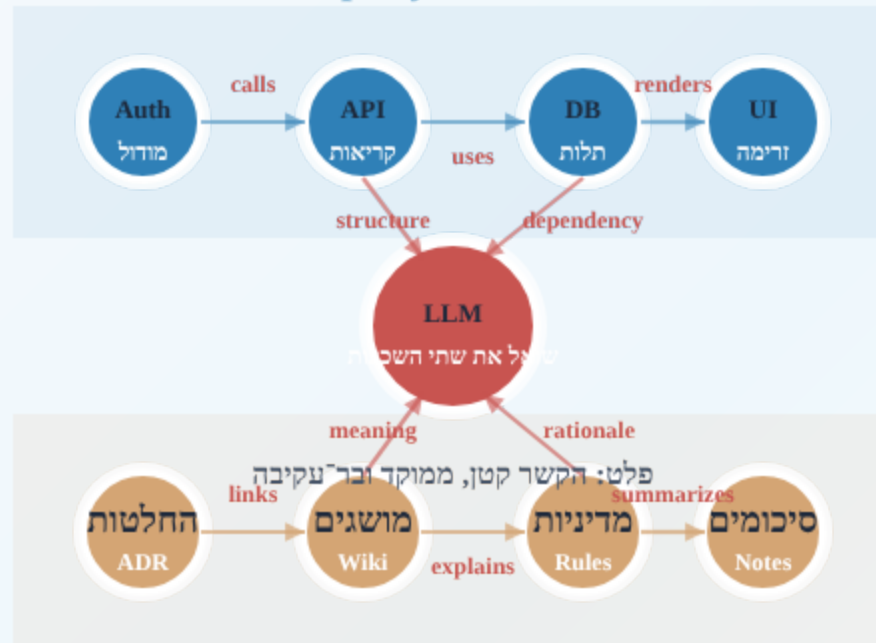
גרף הקוד מציג רכיבים, תלות, זרימות ויחסי קריאה. במקום לקרוא קבצים גולמיים, הסוכן מנווט במפה מבנית.

Node = קובץ, מחלקה, פונקציה או מושג.

Edge = קשר כגון קריאה, שימוש או תלות.

Cluster = אזור פונקציונלי במערכת.

שכבת מבנה: Graphify



שכבת משמעות: LLM Wiki

המודל אינו מקבל “הר טקסט”; הוא מקבל שתי שכבות ממופות: מבנה קוד וידע מעובד.

LLM Wiki שכבת ידע סמנטית

דפי **Markdown** מעובדים מחזיקים מושגים, החלטות, הנחות ומדיניות. זו שכבת זיכרון קריאה לאדם ולמודל.

index.md = אינדקס ניווט קצר.

wiki/ = ידע שעבר סיכום וקישור.

log.md = עקיבות של קליטת ידע.

הגדרה: Skeleton = שלד מבני של מערכת; **Semantic Memory** = זיכרון משמעותי ומקושר; **Traceable Context** = הקשר שאפשר להסביר מאיפה הגיע.

המשך למצגת Graphify: מקריאת גרף יחיד לארכיטקטורת זיכרון שלמה

SKILLS הם שער ההפעלה של הסוכן

הסוכן אינו "זוכר" אוטומטית איך לעבוד; **SKILL** הופך ידע לפרוטוקול הפעלה.

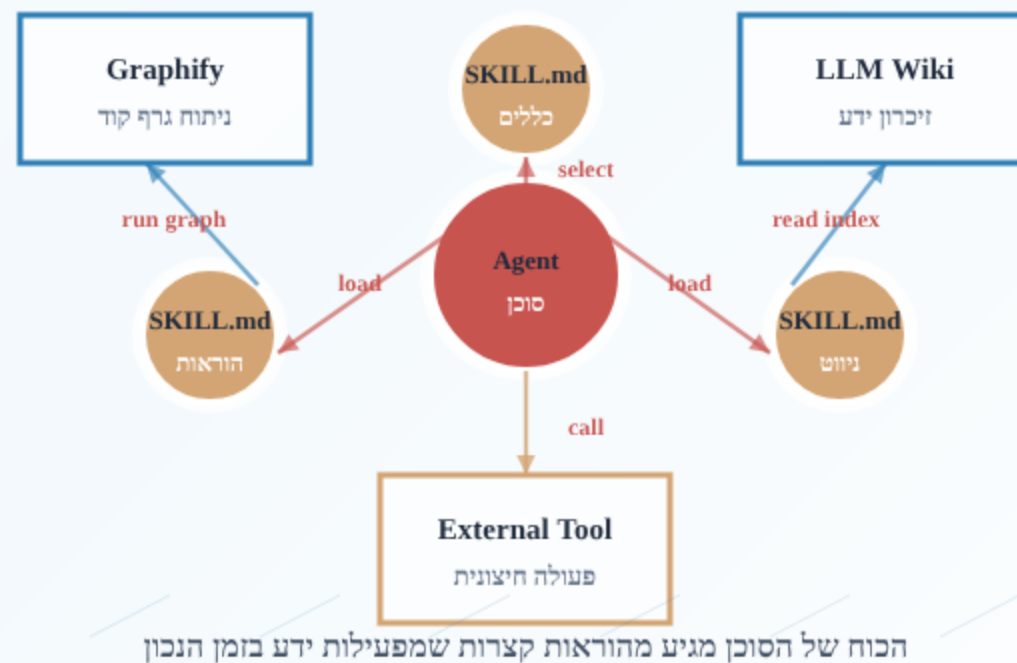
Graphify מספק מפת קוד; ה-**SKILL** קובע מתי להריץ ניתוח גרפי ומה לחלץ מהתוצאה.

LLM Wiki מספק זיכרון סמנטי; ה-**SKILL** מגדיר כיצד לקרוא **index.md** ולבחור דפים רלוונטיים.

Tool = כלי חיצוני או פנימי; **Gateway** = שער שמוודא שהכלי מופעל לפי כללים ולא באקראי.

פרוטוקול: זיהוי משימה → טעינת **SKILL** → קריאת הוראות → הפעלת כלי → החזרת תוצאה ברת-עקיבה.

שער מבוקר בין משימה לכלים



הגדרה: SKILL = חבילת הוראות וכללי שימוש שמלמדת את הסוכן מתי ואיך להפעיל כלי, תהליך או מקור ידע מסוים.

הסקיל אינו “טקסט עזר”; הוא מפרט הפעלה שמחבר בין כוונת המשתמש לבין פעולה נשלטת.

Frontmatter = מטא־דאטה בראש הקובץ, בפורמט קריא למכונה, שמגדיר שם ותיאור.

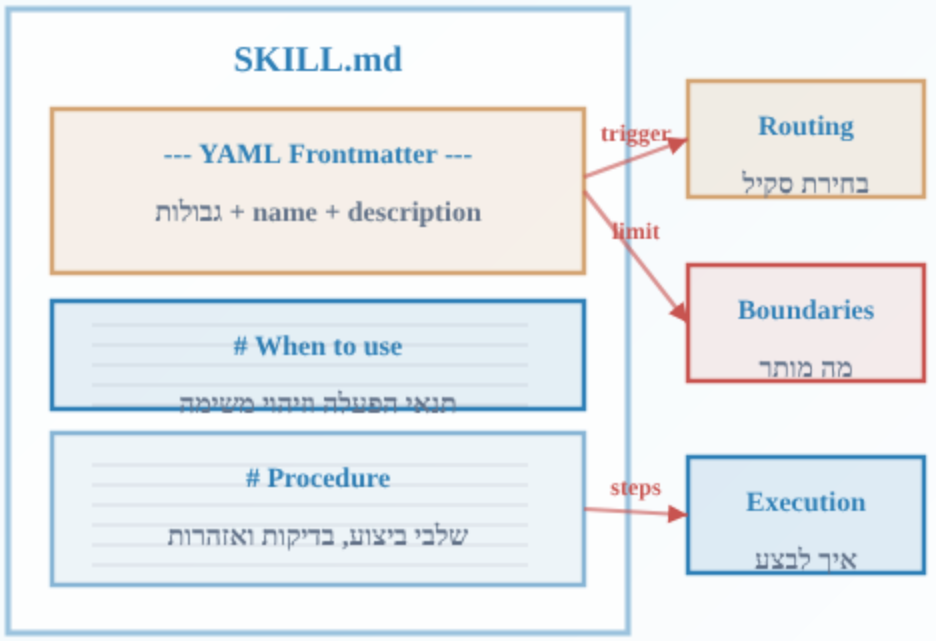
name = מזהה קצר ויציב של הסקיל; **description** = משפט ניתוב שמסביר מתי להשתמש בו.

allowed tools = גבולות תפעוליים: אילו כלים או פעולות מותר להפעיל במסגרת הסקיל.

Markdown body = גוף הוראות בעברית/אנגלית: שלבים, אזהרות, דוגמאות, ותבניות עבודה.

כלל תכנון: תיאור קצר וטוב מפעיל את הסקיל בזמן הנכון; הוראות גוף טובות מונעות אלתור מסוכן.

קובץ סקיל כשרטוט הפעלה



מטא־דאטה מפעיל; גוף ההוראות מכוון

הגדרה: SKILL.md הוא קובץ הנחיות שמלמד סוכן מתי להפעיל יכולת, באילו כלים להשתמש, ובאיזה סדר לבצע את הפעולה.

ככל ש-SKILLS מאפשרים לסוכן לפעול לבד, כך חייבים להגדיר גבולות להפעלה אוטונומית.

Read-only = קריאה בלבד: לרוב בטוח להפעלה אוטונומית, כי אין שינוי במערכת.

Reversible = פעולה הפיכה: מותרת רק אם קיימת דרך ברורה לביטול או שחזור.

Irreversible = פעולה בלתי הפיכה: מחיקה, שליחה, תשלום או פרסום דורשים אישור מפורש.

disable-model-invocation = מנגנון שמונע מהמודל להפעיל מיומנות מסוימת בעצמו; ההפעלה נשארת בשליטת אדם או תהליך מאושר.

כלל בטיחות: ככל שהפעולה משפיעה על אנשים, כסף, נתונים או מוניטין – האוטונומיה מצטמצמת.

מודל הפעלה מבוקר ל-SKILLS



disable-model-invocation: לא כל יכולת זמינה להפעלה עצמית

הגדרה: Guardrails = גבולות פעולה שמגדירים מתי הסוכן רשאי לפעול לבד, מתי עליו לבקש אישור, ומתי פעולה חסומה מראש.

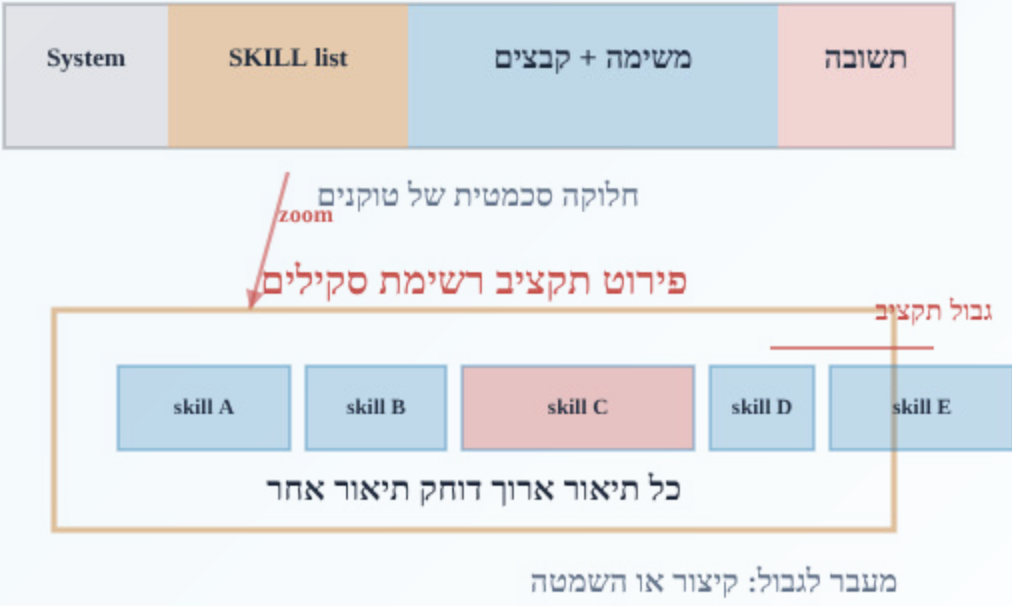
גם רשימת הסקילים צורכת הקשר

רשימת ה-SKILLS איננה “בחינם”: גם היא נכנסת לחלון ההקשר ומתחרה עם משימת המשתמש.

- Skill description = תיאור קצר שמסביר למודל מתי להפעיל מיומנות מסוימת.
- ככל שהתיאורים ארוכים יותר, יותר טוקנים נצרכים לפני תחילת העבודה עצמה.
- התקציב מגן על חלון ההקשר, אך יוצר תחרות: אילו סקילים יוצגו ואילו יידחקו החוצה.
- לכן תיאור טוב צריך להיות קצר, מבחין ומפעיל: מספיק כדי לבחור נכון, לא מספיק כדי להציף.

כלל הנדסי: תיאור סקיל הוא חלק ממנגנון ניתוב – לא מדריך ארוך.

חלון ההקשר כתקציב מוגבל



הגדרה: skillListingBudgetFraction = החלק היחסי מחלון ההקשר שמוקצה להצגת רשימת ה-SKILLS ותיאוריהם, לפני שהמודל בוחר במה להשתמש.

חריגה מתקציב מוחקת יכולות

כאשר רשימת ה-**SKILLS** גדולה מדי, המערכת אינה רק מקצרת טקסט – היא עלולה להעלים יכולות מהמודל.

description הוא התיאור הקצר שבעזרתו המודל מחליט מתי להפעיל מיומנות.

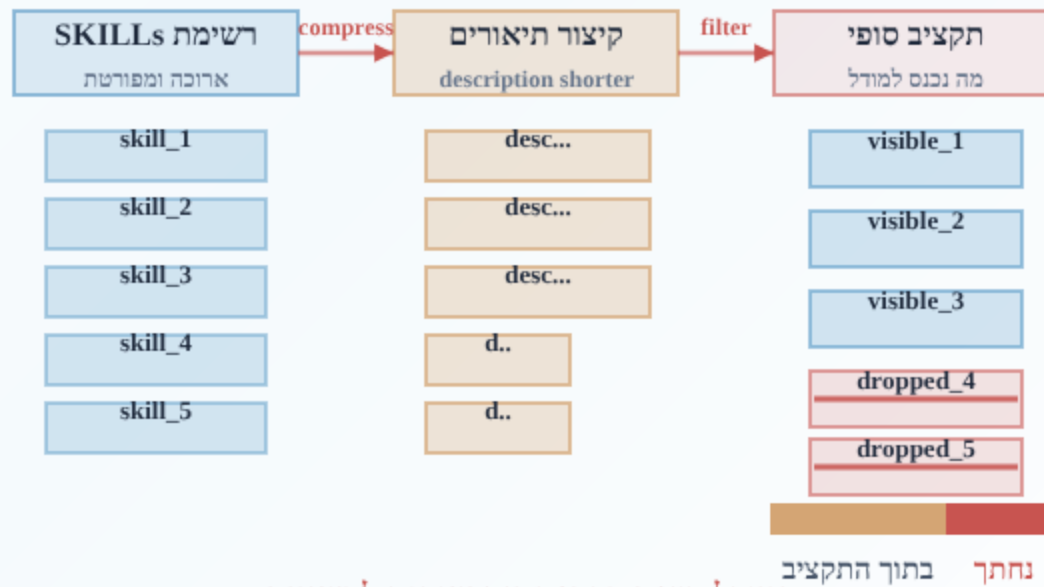
בשלב ראשון התיאורים מתקצרים כדי להיכנס ל-**skillListingBudgetFraction**.

אם עדיין יש חריגה, חלק מה-**SKILLS** מושמטים מהרשימה – ולכן אינם זמינים לבחירה.

הסיכון המעשי: סוכן "שוכח" כלי נכון דווקא במשימה שבה הוא נחוץ.

כלל תכנון: להעדיף מעט **מיומנויות חדות** עם תיאורים מדויקים, במקום ספרייה רחבה של מיומנויות עמומות שצורכות את חלון ההקשר.

תהליך קבלת ההחלטה בתוך תקציב מוגבל



המודל רואה רק את מה שנכנס לרשימה

הגדרה: Skill Dropping = מצב שבו מיומנות קיימת במערכת, אך אינה נכנסת לרשימת היכולות שמוצגת למודל בגלל מגבלת תקציב הקשר.

Lost in the Middle חושף את מגבלת החלון

גם כאשר חלון ההקשר גדול, המודל אינו מתייחס לכל מיקום באותה איכות: **האמצע פגיע במיוחד.**

במחקר **Lost in the Middle**, ביצועי מודלים ירדו כאשר המידע הרלוונטי מוקם באמצע ההקשר.

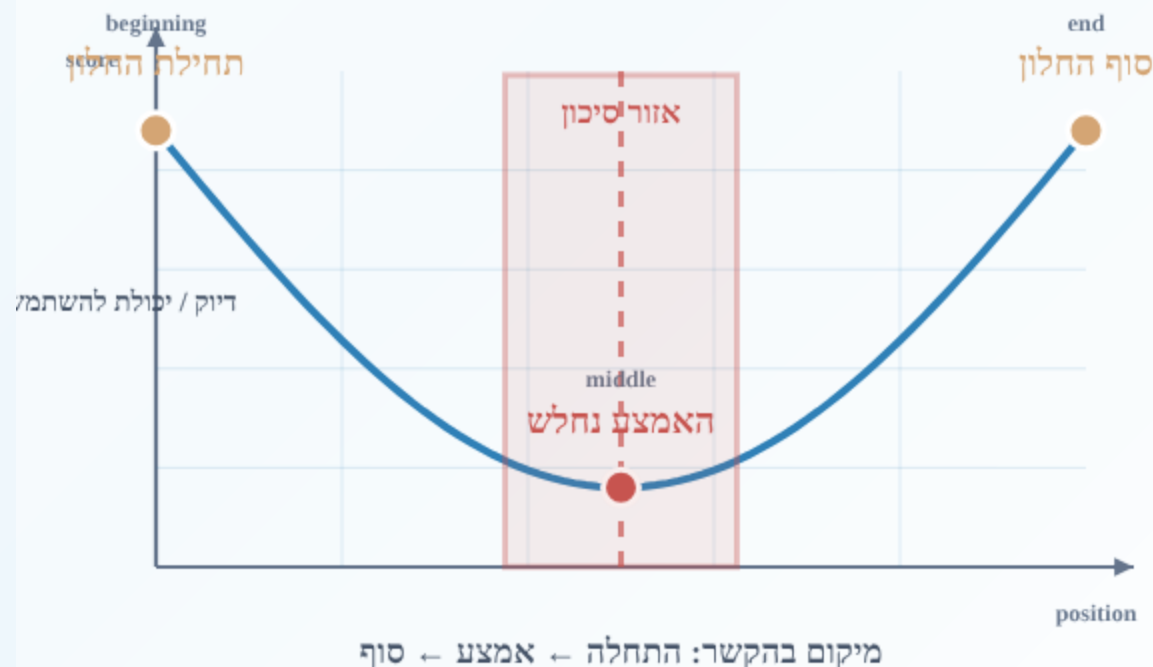
Context Window = כל הטקסט שהמודל רואה ברגע נתון;
Position = מיקום המידע בתוך החלון.

התבנית דומה לעקומת **U**: התחלה וסוף חזקים יותר, המרכז חלש יותר.

לכן הוראות, החלטות ומקורות קריטיים צריכים להופיע בקצוות או להיות משוחזרים לאחר דחיסה.

מסקנה מעשית: ניהול הקשר הוא גם ניהול מיקום – לא רק ניהול כמות טוקנים.

ביצועים כתלות במיקום המידע הרלוונטי



הגדרה: Lost in the Middle = תופעה שבה מודל שפה משתמש טוב יותר במידע שמופיע בתחילת או בסוף חלון ההקשר, ופחות במידע שנמצא באמצע.

Overflow אינו Context Rot

חלון גדול אינו מבטיח תשובה טובה: ההקשר יכול להירקב לפני שהוא נשבר.

Overflow = חריגה ממגבלת חלון ההקשר; המערכת חייבת לקצר, לדחות או לחתוך קלט.

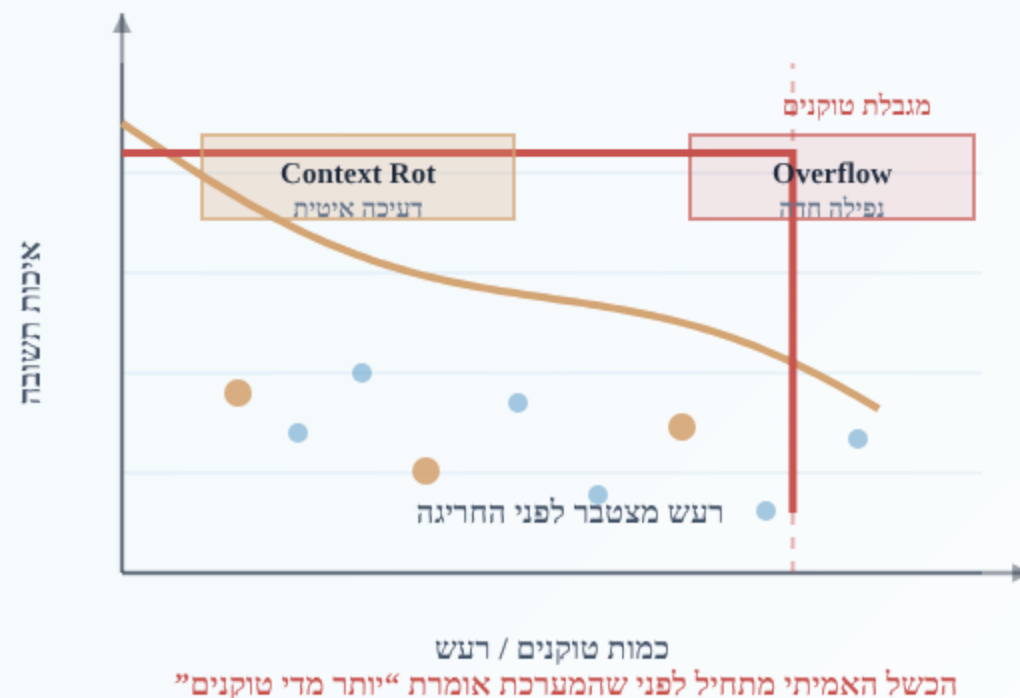
Context Rot = ירידה הדרגתית באיכות ההסקה בגלל רעש, כפילויות ומסיחים סמנטיים.

Attention Dilution = דילול קשב: משאב הקשב מתפזר בין יותר טוקנים, ולכן אות חשוב מקבל פחות משקל.

Semantic Noise = רעש סמנטי: טקסט שנראה קשור לנושא אך אינו תורם להחלטה הנכונה.

כלל הנדסי: הקשר קצר, נקי וממופה עדיף על הקשר ארוך, מלא ורועש.

שני כשלים שונים של חלון ההקשר



הבחנה: Overflow הוא כשל קשיח כשהקלט חורג ממגבלת הטוקנים; **Context Rot** הוא דעיכה הדרגתית באיכות גם כאשר הקלט עדיין "נכנס".

Attention הוא תקציב, לא מחסן

חלון הקשר גדול אינו מבטיח הבנה טובה: המודל עדיין צריך לחלק את ה-Attention בין כל הפריטים שבתוכו.

Attention weights הם משקלי קשב: חלוקה יחסית של חשיבות, לא מקום אחסון בלתי מוגבל.

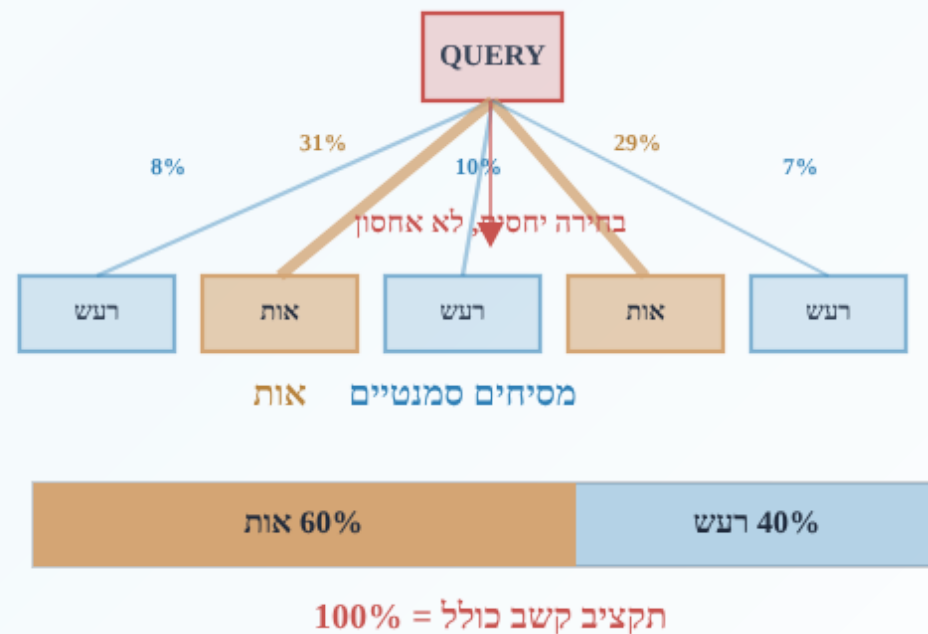
כאשר מוסיפים קטעים דומים אך לא נחוצים, הם מתחרים עם המידע החשוב על אותו תקציב קשב.

Lost in the Middle מתרחש כי מידע קריטי באמצע החלון עלול לקבל פחות משקל אפקטיבי.

Graphify, LLM Wiki ו-SKILLS משפרים את היחס: פחות רעש, יותר אות.

כלל הנדסי: אל תגדילו רק את החלון – הגדילו את שיעור האות בתוך החלון.

חלוקת קשב על פני טוקנים



הגדרה: Self-Attention = מנגנון שבו כל טוקן מקצה משקל יחסי לטוקנים אחרים; **Token** = יחידת טקסט חישובית; **Distractor** = מסיח סמנטי שנראה קשור אך אינו תורם לתשובה.



מה קורה בפועל: בתחילת סשן, הוראות ה־SKILL קרובות לקצה ולכן קל למודל להשתמש בהן. ככל שמתווספים קבצים, שיחות ותוצרים, הן נדחפות לאזור האמצע של **Context Window** — חלון ההקשר.

פתרון תפעולי: לפני פעולה רגישה או מורכבת, מרעננים את ההוראות: קוראים שוב את ה־SKILL, מבצעים **/compact** לדחיסת רעש, או מציבים תקציר כללים קריטיים סמוך לפרומפט הנוכחי.

פתרונות מחקריים מאזנים את המיקום

אם האמצע של ההקשר חלש, הפתרון הוא איזון מיקום: קידוד טוב יותר, קשב מכויל יותר, וסידור חכם יותר.

Multi-Scale Positional Encoding = קידוד מיקום רב-סקאלי: ייצוג מיקום גם ברמה מקומית וגם ברמה רחבה, כדי להפחית תלות במיקום יחיד.

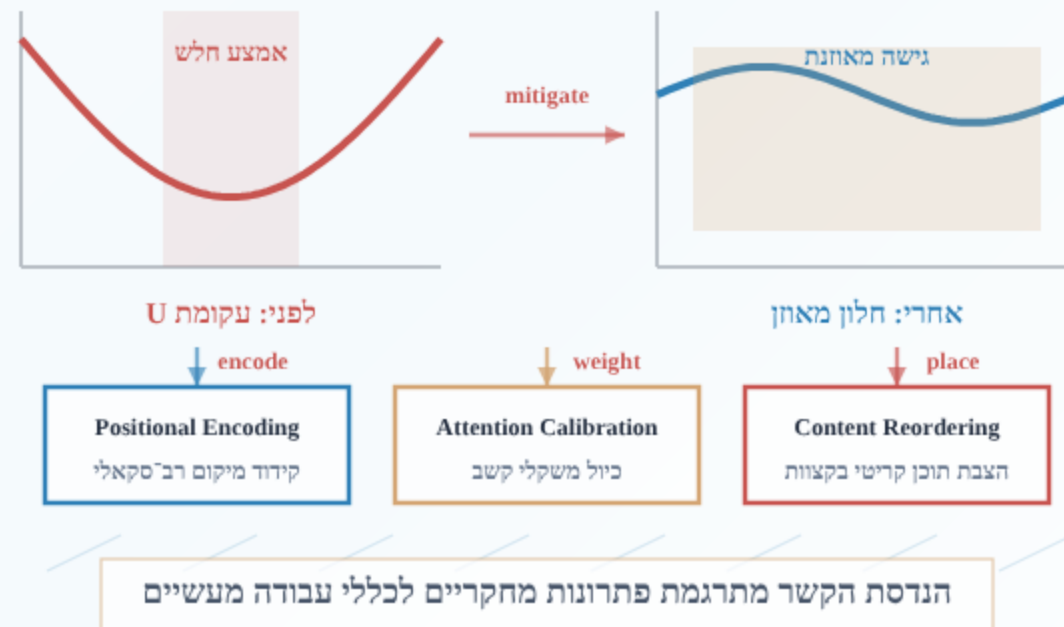
Attention Calibration = כיול קשב: התאמת משקלי הקשב כך שמידע חשוב לא יאבד רק מפני שהוא באמצע החלון.

Reordering = סידור מחדש: הצבת הוראות, ראיות והחלטות חשובות סמוך להתחלה או לסוף.

LLM Wiki, Graphify ו-**SKILLS** מיישמים את העיקרון הנדסי: מפה, זיכרון והוראות במקום הנכון.

כלל מעשי: מידע קריטי צריך להיות גם נכון, גם קצר, וגם ממוקם במקום שהמודל משתמש בו בפועל.

מִרְגִּישוֹת לַמִּיקוּם אֶל עֵיצוֹב מוֹדֵע־מִיקוּם



הגדרה: פתרון מבוסס־מיקום אינו רק “להכניס יותר טקסט”, אלא לשפר את הדרך שבה המודל מקודד, שוקל ומסדר מידע קריטי בתוך חלון ההקשר.

compact/ מחזיר את החשוב לקצוות

דחיסה טובה אינה "מחיקה"; היא ארגון מחדש של הזיכרון הפעיל כדי שהמודל יחזור להשתמש בעיקר.

Context Window = חלון ההקשר: כל הטקסט שהמודל רואה כרגע.

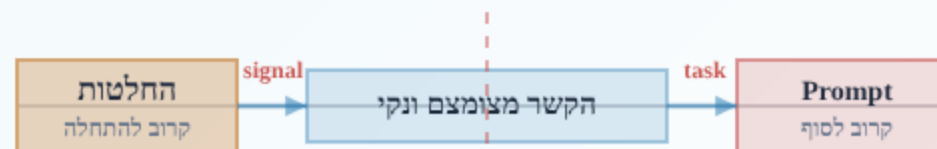
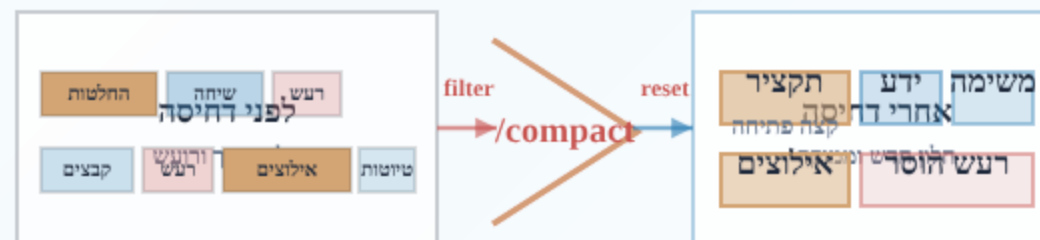
Summary = תקציר מכוון־משימה: מה הוחלט, מה חשוב, ומה אסור לשכוח.

Noise = רעש: חזרות, ניסיונות ביניים, קטעים דומים ומידע שאינו משנה את ההחלטה.

אחרי **compact/**, הכללים הקריטיים חוזרים סמוך לקצה החלון, והמשימה הנוכחית נשארת בקצה השני.

פרוטוקול: דחיסה → בדיקת החלטות שנשמרו → **טעינת SKILL רלוונטי** → המשך עבודה.

מחלון רועש לחלון נקי ומכוון



האמצע כבר אינו מחסן של רעש

הדחיסה מחזירה יחס אות־לרעש גבוה וממקמת מחדש את מה שחשוב

הגדרה: compact/ = פעולת דחיסה שמייצרת חלון הקשר חדש וקצר יותר: החלטות, אילוצים וממצאים נשמרים; רעש, חזרות ופרטים לא רלוונטיים נחתכים.

אתגר מול פתרון: מבט מערכתי

מפת החלטה קצרה: לכל כשל אופייני בחלון ההקשר או בניהול SKILLS יש סימן אזהרה, תגובת תכנון, ובדיקת תפעול פשוטה.

אתגר	מה משתבש	פתרון הנדסי	בדיקת שטח
Overflow חריגה קשיחה ממגבלת טוקנים.	הקלט נחתך, נדחה או נדחס בלי הבחנה; מידע חשוב עלול להיעלם.	תקצוב הקשר : סיכום מוקדם, בחירת מקורות, ומעבר מ"הכול בפנים" ל"רק מה שנחוץ".	אם צריך לחתוך , האם ברור מה נשמר ולמה?
Context Rot דעיכת איכות בגלל רעש.	המודל עדיין מקבל את הטקסט, אך הקשב מתפזר בין כפילויות ומסיחים סמנטיים.	ניקוי אות-רעש : LLM Wiki מתומצת, אינדקסים, והסרת טקסט שאינו תורם להחלטה.	שאלת בקרה : האם כל קטע תורם להחלטה?
Lost in the Middle מידע חשוב נחלש באמצע.	הוראות או ראיות קריטיות קיימות בהקשר, אבל ממוקמות באזור שהמודל משתמש בו פחות.	ניהול מיקום : להציב כללים וקביעות קריטיות בתחילת החלון, בסופו, או אחרי compact .	האם הקריטי נמצא בקצוות ולא קבור באמצע?
SKILLS נעלמים מיומנות קיימת אך לא מוצגת.	תקציב רשימת הסקילים גורם לקיצור תיאורים או להשמטת מיומנויות מהרשימה.	תיאורים חדים : מעט סקילים מובחנים, description קצר ומפעיל, ורענון הסקיל לפני פעולה מורכבת.	האם הסקיל נראה למודל ברגע הבחירה?
SKILLS מסוכנים אוטונומיה ללא גבולות.	הסוכן עשוי להפעיל פעולה רגישה: מחיקה, שליחה, פרסום או שינוי בלתי הפיך.	גבולות הפעלה : Guardrails, אישור אנושי, והרשאות שונות לקריאה, כתיבה ופעולות בלתי הפיכות.	האם פעולה רגישה דורשת אישור מפורש?

זרימת סיכון



המסר המערכתי: ארכיטקטורת ידע טובה אינה מגדילה הקשר בלבד; היא שומרת על קשב, מיקום, הרשאות ועקיבות.

הסינתזה: גרף, ויקי והנדסת הקשר

היעד אינו "עוד הקשר", אלא מערכת שמביאה את המידע הנכון למקום הנכון בזמן הנכון.

Graphify = גרף קוד ותלויות: מי קורא למי, אילו רכיבים קשורים, ואיפה יש מוקדי סיכון.

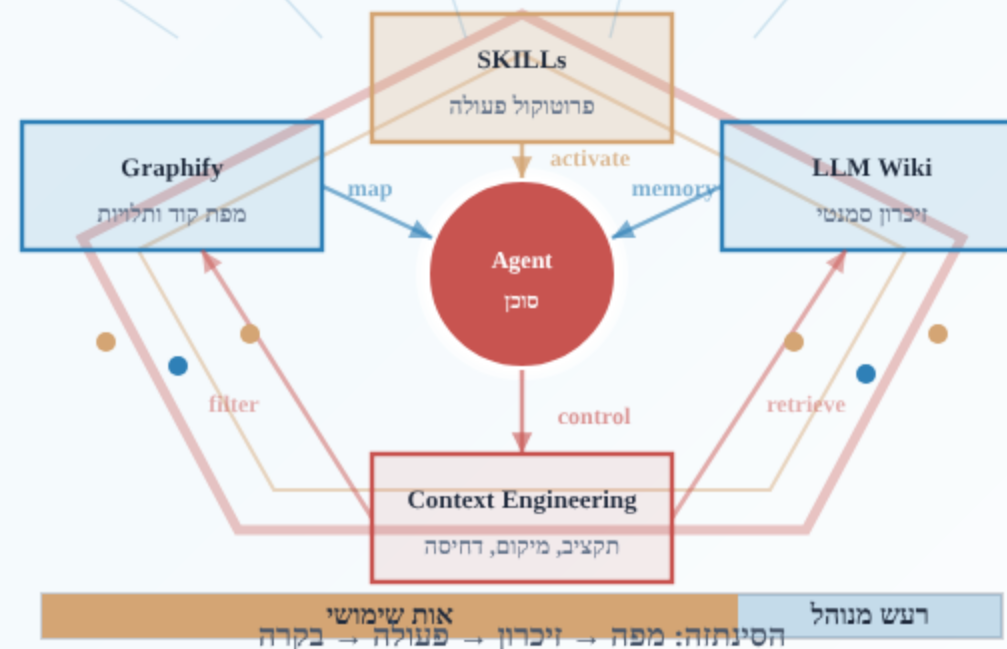
LLM Wiki = ויקי למודל: זיכרון ידע מתומצת, ממיון, ומנוהל לפי דפים ואינדקסים.

SKILLS = מיומנויות הפעלה: הוראות קצרות שמתרגמות ידע לפעולה עקבית ובת-עקיבה.

Context Engineering = הנדסת הקשר: תקצוב, דחיסה, מיקום בקצוות, וסילוק רעש סמנטי.

המדד המאחד: יחס אות-לרעש גבוה – פחות טקסט מיותר, יותר החלטות נכונות.

מערכת אחת לשמירת קשב ועקיבות



הגדרה מסכמת: ארכיטקטורת ידע טובה מחברת בין **Graphify** כמפת מבנה, **LLM Wiki** כזיכרון סמנטי, **SKILLS** כפרוטוקול פעולה, ו**Context Engineering** כבקרת קשב.

ארכיטקטורה טובה מגינה על הקשב

תרגיל מסכם: להפוך מאגר קוד למערכת ידע נשלטת



המטרה אינה תשובה ארוכה יותר — אלא תשובה נכונה, קצרה, עקיבה ובטוחה יותר

1. לבנות נכסי ידע

בחרו ריפו אמיתי והפיקו ממנו מפת **Graphify**, דפי **LLM Wiki**, ואינדקס קצר שמסביר היכן נמצא כל ידע קריטי.

2. להגדיר פרוטוקול פעולה

כתבו **SKILL.md** אחד לפחות: מתי משתמשים בו, אילו כלים מותרים, ומהם **Guardrails** — גבולות פעולה ואישור.

3. למדוד לפני ואחרי

מדדו **Metrics**: עקיבות מקור, ירידת רעש, זיהוי קבצים נכונים, והאם הסוכן מפעיל את הכלי הנכון בזמן הנכון.